

Summary:

1. The use of Google's Gemini Pro model
2. Implementation through the LangChain framework
3. Demonstration of multiple capabilities (text generation, image recognition, and RAG)
4. The specific application of RAG for extracting information from PDFs to answer queries

Keywords:

Artificial intelligence(AI), Deep Learning, Large Language Model(LLM), Generative AI, Retrieval Augmented Generation(RAG)

Terminologies:

1. Large Language Models: Refers to general purpose language models that can be pre-trained and then fine-tuned for specific purposes. Mainly, they are trained to solve common language problems. For example, Text Classification, Question Answering, Document Summarization, Text Generation etc.

2. Generative AI(GenAI): Generative AI is a type of Artificial Intelligence that creates new content based on what it has learnt from existing content.

OR

Generative AI is a Artificial Intelligence technology capable of generating various type of content, including text, imagery, audio, synthetic data.

3. Retrieval Augmented Generation(RAG): RAG is a process of retrieving data from outside a foundation model and argument your prompts, which is a natural language text that requests the LLM to perform a specific task by adding the relevant retrieved data in context.

4. Embedding models: Embedding refers to transforming data like text, images, audio into numerical representation in a high dimensional vector space using Machine Learning(ML) algorithms.

5. LangChain: LangChain is an advanced framework designed for creating applications powered by LLMs. It provides a suit of tools and abstraction to streamline the development process of LLM based application.

6. Prompts: The prompt is a short piece of text that is given to LLM as an input, and it can be used to control the output of the model in a variety of ways.

#code

```
#install necessary dependencies (1. google-generativeai, 2. langchain-google-genai, 3. chrom
!pip install -q --upgrade google-generativeai langchain-google-genai chromadb pypdf
```



```

===== 163.1/163.1 kB 4.3 MB/s eta 0:00:00
===== 559.5/559.5 kB 8.7 MB/s eta 0:00:00
===== 290.4/290.4 kB 10.6 MB/s eta 0:00:00
===== 717.3/717.3 kB 13.3 MB/s eta 0:00:00
===== 321.8/321.8 kB 10.2 MB/s eta 0:00:00
===== 2.4/2.4 MB 22.7 MB/s eta 0:00:00
===== 92.0/92.0 kB 12.1 MB/s eta 0:00:00
===== 62.4/62.4 kB 8.4 MB/s eta 0:00:00
===== 41.3/41.3 kB 4.4 MB/s eta 0:00:00
===== 6.8/6.8 MB 39.7 MB/s eta 0:00:00
===== 59.9/59.9 kB 5.2 MB/s eta 0:00:00
===== 107.0/107.0 kB 13.2 MB/s eta 0:00:00
===== 67.3/67.3 kB 6.7 MB/s eta 0:00:00
```

Installing build dependencies ... done

Getting requirements to build wheel ... done

Preparing metadata (pyproject.toml) ... done

```

===== 283.7/283.7 kB 23.9 MB/s eta 0:00:00
===== 1.7/1.7 MB 41.2 MB/s eta 0:00:00
===== 67.6/67.6 kB 5.9 MB/s eta 0:00:00
===== 145.0/145.0 kB 293.4 kB/s eta 0:00:00
===== 75.6/75.6 kB 7.0 MB/s eta 0:00:00
===== 71.9/71.9 kB 8.5 MB/s eta 0:00:00
===== 53.6/53.6 kB 4.9 MB/s eta 0:00:00
===== 77.9/77.9 kB 9.0 MB/s eta 0:00:00
===== 58.3/58.3 kB 8.3 MB/s eta 0:00:00
===== 127.4/127.4 kB 12.6 MB/s eta 0:00:00
===== 46.0/46.0 kB 5.3 MB/s eta 0:00:00
===== 52.5/52.5 kB 7.0 MB/s eta 0:00:00
===== 130.5/130.5 kB 16.2 MB/s eta 0:00:00
===== 341.4/341.4 kB 32.4 MB/s eta 0:00:00
===== 3.4/3.4 MB 60.4 MB/s eta 0:00:00
===== 1.2/1.2 MB 53.0 MB/s eta 0:00:00
===== 130.2/130.2 kB 15.4 MB/s eta 0:00:00
===== 307.7/307.7 kB 25.2 MB/s eta 0:00:00
===== 86.8/86.8 kB 11.4 MB/s eta 0:00:00
```

Building wheel for pypika (pyproject.toml) ... done

```
#install utility functions for better output visualization(1. display, 2. markdown)
```

```
from IPython.display import display, Markdown
```

```
#import textwrap for formatting and manipulating plane text
```

```
import textwrap
```

```
#create a function which will format the text
```

```
def to_markdown(text):
```

```
    text = text.replace('•', '*')
```

```
    return Markdown(textwrap.indent(text, '>', predicate = lambda _: True))
```

```
# now import genai from google-generativeai
import google.generativeai as genai
#import google collabs inherit library to use google's gemini api key
from google.colab import userdata
```

```
import os
GOOGLE_API_KEY = userdata.get('GOOGLE_API_KEY')
genai.configure(api_key = GOOGLE_API_KEY)
type(GOOGLE_API_KEY)
```

```
→ str
```

```
#lets initialise Google's foundational model and generate some content
model = genai.GenerativeModel(model_name="gemini-pro")
model
```

```
→ genai.GenerativeModel(
    model_name='models/gemini-pro',
    generation_config={},
    safety_settings={},
    tools=None,
    system_instruction=None,
    cached_content=None
)
```

Text Generation

```
#to generate content using Gemini only we can use "response = model.generate.content("pass_c
```

```
#as we're using langchain to build our RAG application again we've to assign Google api key
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(model="gemini-pro", google_api_key=GOOGLE_API_KEY)
```

```
#passing queries to llm
result = llm.invoke("Define LangChain.")
#use markdown function to format generated outcome/result
to_markdown(result.content)
```



LangChain is a large-scale generative language model released by Google AI in 2023. It is designed to understand and generate text in a wide range of languages, including English, Chinese, French, German, Spanish, and Japanese.

Key Features:

- **Multilingual:** Can process and generate text in over 100 languages.
- **Massive Size:** Trained on a dataset of over 138 billion tokens, making it one of the largest language models to date.
- **Advanced Architecture:** Utilizes a Transformer-based neural network architecture with over 175 billion parameters.
- **Wide Applications:** Can be used for various natural language processing tasks, including translation, text summarization, question answering, and dialogue generation.
- **Cross-Lingual Transfer:** Can transfer knowledge and generalize to languages that were not explicitly included in its training data.

Benefits:

- **Enhanced Language Understanding:** Can comprehend complex text and extract meaningful insights.
- **Improved Text Generation:** Generates fluent, grammatically correct text that is indistinguishable from human-written content.
- **Efficient Translation:** Provides high-quality translations between multiple languages.
- **Versatile Tool:** Can be applied to a wide range of applications, from customer service chatbots to document summarization.

Applications:

LangChain has been used in various applications, including:

- Machine Translation

Image Recognition

```
#import python imaging library for image processing
import PIL.Image
img = PIL.Image.open('image.png')
img
```



```
#our aim is to describe this image
#to describe this image We simply need to pass a message
from langchain_core.messages import HumanMessage
#to send messages to and receive responses from a Google Generative AI model within the Lang
from langchain_google_genai import ChatGoogleGenerativeAI
```

```
#lets select relevant model for image processing (gemini vision pro)
llm = ChatGoogleGenerativeAI(model = "gemini-pro-vision", google_api_key=GOOGLE_API_KEY)

#design message and send it to llm to get the response
message = HumanMessage(
    content=[
        {
            "type": "text",
            "text": " Write a short description about the attached image",
        },
        {"type": "image_url", "image_url": "image.png"},
    ]
)
result = llm.invoke([message])
to_markdown(result.content)
```



This is an 8-bit image of a dinosaur. It is white with black outlines. It is standing on two legs and has its arms in the air. Its mouth is open and it has a surprised expression on its face. It is the icon that is displayed in the Google Chrome web browser when there is no internet connection.

Interact with document using Retrieval Augmented Generation(RAG)

```
#To perform operation on document, In our case PDF
#1. We will divide the pdf into chunk of text
#2. Then we will generate the embeddings of these chunk of text using Google generative AI t
#3. All the generated text embeddings will be stored into vector database
#4. Then the query will be performed and embedded
#5. After that similarity/symantic search will be performed using vector database's similari
#6. Retriever will retrieve set of text or a sentences which are similar or matches context
#7. This retrieved data/text/sentences will be passed to the LLM along with actual query/que
#8. Lastly LLM will generate concise and grammatically correct answer for that respective qu
```

```
#In order to work with RAG we need to install respective dependencies
#Tesseract OCR is an open-source Optical Character Recognition (OCR) engine maintained by Gc
#It is widely used for extracting text from images, including scanned documents, photographs
#Tesseract OCR is highly accurate and supports various languages and file formats.
```

```
!sudo apt -y -qq install tesseract-ocr libtesseract-dev
```

```
!sudo apt-get -y -qq install poppler-utils libxml2-dev libxslt1-dev antiword unrar poppler-u
```

```
!pip install langchain
```



```

Setting up libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.7) ...
Setting up ghostscript (9.55.0~dfsg1-0ubuntu5.7) ...
Setting up sox (14.4.2+git20190427-2+deb11u2ubuntu0.22.04.1) ...
Setting up pstotext (1.9-6build1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.4) ...
/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

Processing triggers for man-db (2.10.2-1) ...
Processing triggers for fontconfig (2.13.1-4.2ubuntu5) ...
Collecting langchain
  Downloading langchain-0.2.5-py3-none-any.whl (974 kB)
    974.6/974.6 kB 8.3 MB/s eta 0:00:00
Requirement already satisfied: PyYAML>=5.3 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: SQLAlchemy<3,>=1.4 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: aiohttp<4.0.0,>=3.8.3 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: async-timeout<5.0.0,>=4.0.0 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: langchain-core<0.3.0,>=0.2.7 in /usr/local/lib/python3.10/dist-packages
Collecting langchain-text-splitters<0.3.0,>=0.2.0 (from langchain)
  Downloading langchain_text_splitters-0.2.1-py3-none-any.whl (23 kB)
Requirement already satisfied: langsmith<0.2.0,>=0.1.17 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: numpy<2,>=1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: pydantic<3,>=1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: requests<3,>=2 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: tenacity<9.0.0,>=8.1.0 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: yarl<2.0,>=1.0 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: jsonpatch<2.0,>=1.33 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: packaging<25,>=23.2 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: orjson<4.0.0,>=3.9.14 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: annotated-types>=0.4.0 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: pydantic-core==2.18.4 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: typing-extensions>=4.6.1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: greenlet!=0.4.17 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.10/dist-packages
Installing collected packages: langchain-text-splitters, langchain

```

```
!pip install langchain_community
```

```
#Useful for fetching data from URLs, parsing URLs, and handling web-related operations withi
```

```

import urllib
#Helpful for managing and responding to warnings generated during program execution, ensuring
import warnings
#pathlib provides an object-oriented approach to working with filesystem paths in Python
from pathlib import Path as p
#pprint enhances the output of data structures, improving readability for complex nested objects
from pprint import pprint

import pandas as pd
#PromptTemplate class is used for defining and managing templates or patterns for generating
#It includes methods for formatting and customizing prompts based on variables or conditions
from langchain import PromptTemplate
#load_qa_chain this function is expected to handle the loading or initialization of a question
#which is a specific configuration or pipeline for processing and responding to questions.
from langchain.chains.question_answering import load_qa_chain
#for efficiently loading and extracting text from PDF documents
from langchain_community.document_loaders import PyPDFLoader
#for recursively splitting text into smaller segments based on characters or other criteria
from langchain.text_splitter import RecursiveCharacterTextSplitter
#vectorstores contains classes or functions related to storing and managing vector representations
#Chroma handles the storage, retrieval, and management of vector data in an efficient manner
from langchain.vectorstores import Chroma
#RetrievalQA combines retrieval mechanisms with question-answering capabilities
#enabling the system to find and provide answers from a knowledge base or dataset
from langchain.chains import RetrievalQA
#filterwarnings function allows you to specify which warnings should be ignored, displayed,
warnings.filterwarnings("ignore")

```



Collecting langchain_community

Downloading langchain_community-0.2.5-py3-none-any.whl (2.2 MB)

2.2/2.2 MB 17.3 MB/s eta 0:00:00

Requirement already satisfied: PyYAML<=5.3 in /usr/local/lib/python3.10/dist-packages (from langchain_community==0.2.5) (5.3.1)
 Requirement already satisfied: SQLAlchemy<=1.4 in /usr/local/lib/python3.10/dist-packages (from langchain_community==0.2.5) (1.4.4)
 Requirement already satisfied: aiohttp<4.0.0,>=3.8.3 in /usr/local/lib/python3.10/dist-packages (from langchain_community==0.2.5) (3.8.6)
 Collecting dataclasses-json<0.7,>=0.5.7 (from langchain_community)

Downloading dataclasses_json-0.6.7-py3-none-any.whl (28 kB)

Requirement already satisfied: langchain<0.3.0,>=0.2.5 in /usr/local/lib/python3.10/dist-packages (from dataclasses_json==0.6.7) (0.2.11)
 Requirement already satisfied: langchain-core<0.3.0,>=0.2.7 in /usr/local/lib/python3.10/dist-packages (from langchain==0.2.11) (0.2.11)
 Requirement already satisfied: langsmith<0.2.0,>=0.1.0 in /usr/local/lib/python3.10/dist-packages (from langchain==0.2.11) (0.1.10)
 Requirement already satisfied: numpy<2,>=1 in /usr/local/lib/python3.10/dist-packages (from langchain==0.2.11) (1.24.3)
 Requirement already satisfied: requests<3,>=2 in /usr/local/lib/python3.10/dist-packages (from langchain==0.2.11) (2.31.0)
 Requirement already satisfied: tenacity<9.0.0,>=8.1.0 in /usr/local/lib/python3.10/dist-packages (from langchain==0.2.11) (8.1.0)
 Requirement already satisfied: aiosignal<=1.1.2 in /usr/local/lib/python3.10/dist-packages (from aiohttp==3.8.6) (1.1.2)
 Requirement already satisfied: attrs<=17.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp==3.8.6) (17.3.0)
 Requirement already satisfied: frozenlist<=1.1.1 in /usr/local/lib/python3.10/dist-packages (from aiohttp==3.8.6) (1.1.1)
 Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.10/dist-packages (from aiohttp==3.8.6) (4.5.2)
 Requirement already satisfied: yarl<2.0,>=1.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp==3.8.6) (1.0.0)
 Requirement already satisfied: async-timeout<5.0,>=4.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp==3.8.6) (4.0.3)
 Collecting marshmallow<4.0.0,>=3.18.0 (from dataclasses-json<0.7,>=0.5.7->langchain_community==0.2.5)

Downloading marshmallow-3.21.3-py3-none-any.whl (49 kB)

49.2/49.2 kB 7.1 MB/s eta 0:00:00

Collecting typing-inspect<1,>=0.4.0 (from dataclasses-json<0.7,>=0.5.7->langchain_community==0.2.5)

Downloading typing_inspect-0.9.0-py3-none-any.whl (8.8 kB)

```
Requirement already satisfied: langchain-text-splitters<0.3.0,>=0.2.0 in /usr/local/lib/
Requirement already satisfied: pydantic<3,>=1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: jsonpatch<2.0,>=1.33 in /usr/local/lib/python3.10/dist-pa
Requirement already satisfied: packaging<25,>=23.2 in /usr/local/lib/python3.10/dist-pac
Requirement already satisfied: orjson<4.0.0,>=3.9.14 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dis
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-pack
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-pack
Requirement already satisfied: typing-extensions>=4.6.0 in /usr/local/lib/python3.10/dis
Requirement already satisfied: greenlet!=0.4.17 in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: annotated-types>=0.4.0 in /usr/local/lib/python3.10/dist-
Requirement already satisfied: pydantic-core==2.18.4 in /usr/local/lib/python3.10/dist-p
Collecting mypy-extensions>=0.3.0 (from typing-inspect<1,>=0.4.0->dataclasses-json<0.7,>
  Downloading mypy_extensions-1.0.0-py3-none-any.whl (4.7 kB)
Installing collected packages: mypy-extensions, marshmallow, typing-inspect, dataclasses
Successfully installed dataclasses-json-0.6.7 langchain_community-0.2.5 marshmallow-3.21
```

```
import os
GOOGLE_API_KEY = userdata.get('GOOGLE_API_KEY')
genai.configure(api_key = GOOGLE_API_KEY)
type(GOOGLE_API_KEY)
```

```
#install, select model and api key
!pip install langchain-google-genai
from langchain_google_genai import ChatGoogleGenerativeAI
model = ChatGoogleGenerativeAI(model = "gemini-pro", google_api_key = GOOGLE_API_KEY, temper
#temperature controls the randomness of the data and convert_system_message_to_human convert
```

```
➡ Requirement already satisfied: langchain-google-genai in /usr/local/lib/python3.10/dist-
Requirement already satisfied: google-generativeai<0.8.0,>=0.7.0 in /usr/local/lib/pythc
Requirement already satisfied: langchain-core<0.3,>=0.2.9 in /usr/local/lib/python3.10/c
Requirement already satisfied: google-ai-generativelanguage==0.6.5 in /usr/local/lib/pyt
Requirement already satisfied: google-api-core in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: google-auth>=2.15.0 in /usr/local/lib/python3.10/dist-pac
Requirement already satisfied: protobuf in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: pydantic in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from goc
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.10/dist-packa
Requirement already satisfied: proto-plus<2.0.0dev,>=1.22.3 in /usr/local/lib/python3.10
Requirement already satisfied: PyYAML>=5.3 in /usr/local/lib/python3.10/dist-packages (f
Requirement already satisfied: jsonpatch<2.0,>=1.33 in /usr/local/lib/python3.10/dist-pa
Requirement already satisfied: langsmith<0.2.0,>=0.1.75 in /usr/local/lib/python3.10/dis
Requirement already satisfied: packaging<25,>=23.2 in /usr/local/lib/python3.10/dist-pac
Requirement already satisfied: tenacity!=8.4.0,<9.0.0,>=8.1.0 in /usr/local/lib/python3.
Requirement already satisfied: cachetools<6.0,>=2.0.0 in /usr/local/lib/python3.10/dist-
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: orjson<4.0.0,>=3.9.14 in /usr/local/lib/python3.10/dist-p
```

```
Requirement already satisfied: requests<3,>=2 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: annotated-types>=0.4.0 in /usr/local/lib/python3.10/dist-
Requirement already satisfied: pydantic-core==2.18.4 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: googleapis-common-protos<2.0.dev0,>=1.56.2 in /usr/local/
Requirement already satisfied: httplib2<1dev,>=0.15.0 in /usr/local/lib/python3.10/dist-
Requirement already satisfied: google-auth-httplib2>=0.1.0 in /usr/local/lib/python3.10/
Requirement already satisfied: uritemplate<5,>=3.0.1 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: grpcio<2.0dev,>=1.33.2 in /usr/local/lib/python3.10/dist-
Requirement already satisfied: grpcio-status<2.0.dev0,>=1.33.2 in /usr/local/lib/python3
Requirement already satisfied: pyparsing!=3.0.0,!3.0.1,!3.0.2,!3.0.3,<4,>=2.4.2 in /u
Requirement already satisfied: pyasn1<0.7.0,>=0.4.6 in /usr/local/lib/python3.10/dist-pa
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dis
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-pack
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-pack
```

Extract text from PDF

```
pdf_loader = PyPDFLoader("/content/NIPS-2017-attention-is-all-you-need-Paper.pdf")
pages = pdf_loader.load_and_split()
print(pages[3].page_content)
```



Figure 1: The Transformer - model architecture.

wise fully connected feed-forward network. We employ a residual connection [10] around the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the input and output layers, produce outputs of dimension $d_{\text{model}} = 512$.

Decoder: The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions by applying mask padding, combined with fact that the output embeddings are offset by one position, ensuring predictions for position i can depend only on the known outputs at positions less than i .

3.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

3.2.1 Scaled Dot-Product Attention

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot product of the queries with the keys, scaled by $1/\sqrt{d_k}$, and then pass the result through a softmax function to get the attention weights. We then multiply the values by the attention weights to get the output.



```
len(pages)
```



12

RAG Pipeline : Embeddings + Gemini(LLM)

```

from langchain_google_genai import GoogleGenerativeAIEmbeddings

text_splitter = RecursiveCharacterTextSplitter(chunk_size = 10000, chunk_overlap = 1000)
context = "\n\n".join(str(p.page_content) for p in pages)
texts = text_splitter.split_text(context)
#RecursiveCharacterTextSplitter class is responsible for splitting text into smaller chunks
#chunk_size = 10000 means that each chunk will contain up to 10,000 characters
#chunk_overlap = 1000 means that there will be a 1,000 character overlap between each chunk.

#initializing an instance of GoogleGenerativeAIEmbeddings with specific parameters
embeddings = GoogleGenerativeAIEmbeddings(model = "models/embedding-001", google_api_key = G

#use vector database and pass all the chunk of text and embedding model
vector_index = Chroma.from_texts(texts, embeddings).as_retriever(search_kwargs = {"k": 4})
#search_kwargs = {"k": 8} tells while doing similarity search only return 8 documents not mc

#RetrievalQA = Retrieval question answer
#allows for the effective generation of answers to queries, along with the retrieval of rele
qa_chain = RetrievalQA.from_chain_type(model, chain_type = "stuff", retriever = vector_index

#a simple question/query is passed to qa_chain and then closest documents have been retrieve
question = "What is the main focus of the paper? and What is multi-head attention layer?"
result = qa_chain({"query": question})
Markdown(result ["result"])

```



Main Focus of the Paper

The main focus of the paper is to introduce the Transformer, a novel neural network architecture for sequence transduction tasks that relies solely on attention mechanisms, dispensing with recurrence and convolutions entirely. The Transformer achieves state-of-the-art results on machine translation tasks, outperforming existing models in both quality and efficiency.

Multi-Head Attention Layer

A multi-head attention layer is a type of attention mechanism that consists of several parallel attention heads. Each attention head computes a weighted sum of the values, where the weights are computed by a compatibility function of the query with the corresponding key. The outputs of the attention heads are then concatenated and projected to produce the final output of the multi-head attention layer.

Multi-head attention allows the model to jointly attend to information from different representation subspaces

```

#fetch source documents for above answer
result["source_documents"]

```



[Document(page_content='Scaled Dot-Product Attention\n Multi-Head Attention\nFigure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several\nattention layers running in parallel.\nquery with all keys, divide each by\ndk, and apply a softmax function to obtain the weights on the\nvalues.\nIn practice, we compute the attention function on a set of queries simultaneously, packed together\ninto a matrix Q. The keys and values are also packed together into

matrices K and V . We compute the matrix of outputs as: $\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$ (1) The two most commonly used attention functions are additive attention [2], and dot-product (multi- $\sqrt{d_k}$) multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code. While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [3]. We suspect that for large values of d_k , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients⁴. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

3.2.2 Multi-Head Attention

Instead of performing a single attention function with d_{model} -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k, d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2. Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^d q_i k_i$, has mean 0 and variance d_k .

3.2.3 Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways:

- In "encoder-decoder attention" layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [31, 2, 8].
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.
- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections. See Figure 2.

3.3 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer.

#let's design a custom prompt using template in order to generate the answer

```
template = """1. Use the following pieces of context to answer the question at the end.
2. If you have no idea or don't know the answer, just say that this knowledge is out of my
```

3. Keep the results as precise as possible. Always say " Anything else I can help you with:
{context}

Question: {question}

Helpful Answer:"""

QA_CHAIN_PROMPT = PromptTemplate.from_template(template)#passing generated prompt

```
qa_chain = RetrievalQA.from_chain_type(
    model,
    retriever=vector_index,
    return_source_documents=True,
    chain_type_kwargs={"prompt": QA_CHAIN_PROMPT}
)
```

```
#now pass the same question as asked earlier to the RAG model but with our custom prompt
question = "What is the main focus of the paper? and What is multi-head attention layer?"
result = qa_chain({"query": question})
Markdown(result ["result"])
```



1. The main focus of the paper is to introduce the Transformer, a novel neural network architecture for sequence transduction tasks that relies solely on attention mechanisms, dispensing with recurrence and convolutions entirely. The Transformer achieves state-of-the-art results on two machine translation tasks, demonstrating its superiority in quality while being more parallelizable and requiring significantly less time to train.
2. Multi-head attention layer is a key component of the Transformer architecture. It consists of multiple attention heads, each of which performs a scaled dot-product attention operation on different linear projections of the queries, keys, and values. The outputs of the attention heads are then concatenated and projected to obtain the final output of the multi-head attention layer. This mechanism allows the

```
#check if it provides results outside the context/source data
question = "What is Overfitting and underfitting?"
result = qa_chain({"query": question})
Markdown(result ["result"])
```



This knowledge is out of my context, I apologize for not being able to answer this question. Anything else I can help you with?

EOF